Trabajo Práctico 3 - Lógica de Negocio

Ejercicio 1: Lógica de Negocio como Funciones Puras

Objetivo: Implementar y probar reglas de negocio como funciones de Go puras, sin dependencias de frameworks web.

1. Crea un nuevo directorio para el TP y dentro un subdirectorio `logic`.
2. Dentro de `logic`, crea un archivo `products.go`. En este archivo, define la estructura `Product` y las siguientes funciones:
   - `ValidateProduct(p Product) error`: Valida que un producto tenga un nombre no vacío y un precio mayor a cero. Devuelve un `error` si la validación falla.
   - `ApplyDiscount(p Product, percentage float64) Product`: Aplica un descuento porcentual al precio del producto y devuelve el producto modificado.
3. En el directorio raíz del TP, crea un `main.go` que importe el paquete `logic`, cree algunos productos, y use las funciones `ValidateProduct` y `ApplyDiscount` para verificar su funcionamiento, imprimiendo los resultados en la consola.

Ejercicio 2: Exponiendo Lógica con una API REST Básica

Objetivo: Exponer la lógica de negocio a través de una API REST simple que maneje JSON.

1. Tomando el código del ejercicio anterior, modifica `main.go` para crear un servidor HTTP.
2. Usa una variable global (slice) para mantener una lista de productos en memoria.
3. Implementa un endpoint `GET /products` que:
   - Devuelva la lista completa de productos.
   - Codifique la respuesta en formato JSON.
   - Establezca la cabecera `Content-Type` a `application/json`.
4. Implementa un endpoint `POST /products` que:
   - Decodifique un producto en formato JSON del cuerpo de la petición.
   - Use la función `logic.ValidateProduct` para validar los datos recibidos. Si son inválidos, debe devolver un `http.StatusBadRequest` (400) con un mensaje de error.
   - Si es válido, agregue el nuevo producto al slice en memoria.
   - Devuelva el producto recién creado con un código de estado `http.StatusCreated` (201).

Ejercicio 3: API REST con CRUD Completo

Objetivo: Extender la API para soportar todas las operaciones CRUD (Create, Read, Update, Delete).

1. Implementa un endpoint `GET /products/{id}` que:
   - Reciba un ID de producto desde la URL.
   - Busque el producto en el slice.
   - Si lo encuentra, lo devuelva como JSON con estado 200.
   - Si no lo encuentra, devuelva un `http.StatusNotFound` (404).
2. Implementa un endpoint `PUT /products/{id}` que:
   - Reciba un ID y datos de producto en el body.
   - Valide los nuevos datos.
   - Busque y actualice el producto correspondiente.
   - Devuelva el producto actualizado o un error 404 si no existe.
3. Implementa un endpoint `DELETE /products/{id}` que:
   - Elimine un producto por su ID.
   - Devuelva un `http.StatusNoContent` (204) si tiene éxito o un 404 si no existe.
4. **Refactorización**: Crea un único handler para `/products/` que direccione a las funciones correctas (`getProduct`, `updateProduct`, `deleteProduct`) según el método HTTP y la presencia de un ID en la URL.

Ejercicio 4: Middleware y Robustez

Objetivo: Mejorar la API añadiendo funcionalidades transversales como logging y manejo de concurrencia.

1. **Middleware de Logging**:
   - Crea una función `loggingMiddleware` que reciba un `http.Handler` y devuelva un `http.Handler`.
   - Este middleware debe imprimir en consola el método HTTP, la URL y la dirección IP remota de cada petición antes de pasarla al siguiente handler.
   - Aplica este middleware a todos tus endpoints.

2. **Manejo de Concurrencia**:
   - Dado que el slice de productos es un recurso compartido, protégelo de condiciones de carrera (race conditions) al leerlo y escribirlo.
   - Utiliza un `sync.RWMutex` para controlar el acceso concurrente a la lista de productos en todos los handlers.

Recursos Adicionales

- Documentación net/http[1]
- Documentación encoding/json[2]
- Documentación sync[3]
- RESTful API Design: Best Practices[4]
- HTTP Status Codes[5]

Trabajo de Cursada: Exponiendo la Lógica como API

Objetivo: Construir una API REST para gestionar las entidades de tu aplicación, conectando la lógica de negocio con la capa de datos.

Ahora que tienes la capa de datos generada por `sqlc`, la expondrás a través de endpoints HTTP.

1. **Conexión con la Base de Datos**:
   - En tu `main.go`, establece la conexión con tu base de datos (PostgreSQL o SQLite).
   - Crea una instancia del repositorio generado por `sqlc` para poder usarlo en tus handlers.
2. **Implementación de Handlers CRUD**:
   - Crea un handler para cada una de las operaciones CRUD de tu entidad.
   - `POST /<entidades>`: Debe recibir datos en formato JSON, validarlos (ej. que el título no esté vacío), usar el método `Create...` de `sqlc` para guardarlos en la base de datos y devolver el nuevo objeto como JSON con estado 201.
   - `GET /<entidades>`: Debe usar el método `List...` de `sqlc` para obtener todos los registros y devolverlos como un array JSON.
   - `GET /<entidades>/{id}`: Debe obtener el ID de la URL, buscar el registro con `Get...` y devolverlo. Si no existe, debe devolver un 404.
   - `PUT /<entidades>/{id}`: Debe recibir datos JSON, validarlos, y actualizar el registro correspondiente usando `Update...`.
   - `DELETE /<entidades>/{id}`: Debe eliminar el registro usando `Delete...` y devolver un estado 204 (No Content).
3. **Prueba de la API**:
   - Utiliza `curl` o una herramienta como HURL[6] para probar cada uno de los endpoints. Verifica que puedes crear, listar, ver, actualizar y eliminar entidades de tu aplicación.
   - Crea un archivo `requests.{bash o hurl}` con ejemplos de todas las peticiones que has probado.

El entregable debe incluir las instrucciones para ejecutar el servidor y probar la API, así como el código fuente completo del proyecto. Utilizar docker para para facilitar la ejecución en diferentes entornos. Las pruebas de la API deben ser claras y reproducibles a través de los archivos de ejemplo proporcionados con HURL o un script bash que utilice `curl`.

[1] https://pkg.go.dev/net/http
[2] https://pkg.go.dev/encoding/json
[3] https://pkg.go.dev/sync
[4] https://restfulapi.net/resource-naming/
[5] https://httpstatuses.com/
[6] https://hurl.dev/